Calling Agent Demo

Build Plan & Technical Guide

Full working demo in 4 days · ~20–25 hours total · Free-tier stack

Overview

This guide provides a step-by-step plan to build a functional AI-powered calling agent demo using free-tier services. Each phase includes primary tool recommendations along with alternatives, so you can swap components based on availability or preference.

Attribute	Value	Notes
Total Time	20–25 hours	Assuming no major blockers
Timeline	4 days	Continuous, focused work
Cost	$0 (free-tier throughout)	Twilio provides $15 trial credit
Backend	Python (FastAPI) or Node.js	Either works, FastAPI slightly simpler

System Architecture

The calling agent follows a linear audio pipeline. Each stage can be swapped independently:

Caller	Twilio Phone Number	FastAPI / Node Server	STT Engine	LLM (Claude / GPT-4)	TTS Engine	Caller hears response

Each arrow in the pipeline is a WebSocket or HTTP connection. Twilio streams raw audio (8kHz mulaw) to your server via Media Streams. Your backend handles the STT → LLM → TTS loop and streams converted audio back.

Tool Stack with Alternatives

Every layer has a primary recommendation (free-tier) and two alternatives. You can mix and match across stages.

Layer	Primary Tool (Free)	Alternative 1	Alternative 2
Phone / Voice	Twilio (trial credit)	Vonage / Nexmo (free tier)	Plivo ($0.005/min, no setup fee)
Local Tunnel	ngrok (free, 1 tunnel)	localtunnel (fully free)	Cloudflare Tunnel (free)
Backend	FastAPI + Python	Node.js + Express	Flask (Python, simpler)
STT	Deepgram (200 hrs/month)	OpenAI Whisper (self-hosted)	AssemblyAI (5 hrs free/month)
LLM	OpenAI GPT-4o (trial credits)	Claude API (Anthropic)	Groq (fast inference, free tier)
TTS	ElevenLabs (10k chars/month)	Google TTS (1M chars/month)	Amazon Polly (5M chars free/yr)
Hosting	Render (free tier)	Railway (free tier, $5 credit)	Fly.io (free tier, global edge)
Logging / DB	Local JSON file (zero setup)	Supabase (free tier, Postgres)	PlanetScale (free MySQL tier)

Phase 1 — Environment Setup

Get all services connected. By the end of this phase you should be able to dial your Twilio number and hear a static AI greeting.

Day 1	**Environment Setup**
	Goal: All services connect. Phone call returns a static response.
	• Create Twilio account → get a free trial phone number
	• Set up FastAPI (Python) or Express (Node.js) backend
	• Configure webhook endpoint: POST /voice
	• Expose local server with ngrok (or localtunnel as alternative)
	• Connect LLM API key — OpenAI, Claude, or Groq
	• Return a static TwiML <Say> response on call
	✓ **Milestone:** Caller dials the number → agent responds: “Hello, this is your AI assistant.”
	⚡ **Alt:** Shortcut: Use Twilio <Gather> + <Say> with built-in Polly voices to skip custom STT/TTS on Day 1 — swap in better voices on Day 3.

Key Code Snippet — Day 1 Webhook

```
# FastAPI example
@app.post('/voice')
def voice():
    return '<Response><Say>Hello from your AI agent.</Say></Response>'
```

Phase 2 — Speech Pipeline (STT)

Enable real conversation by capturing the caller's speech, transcribing it, and generating an intelligent text response from the LLM.

Day 2	**Speech-to-Text Pipeline**
	Goal: User speaks → AI transcribes → LLM generates a relevant text response.
	• Enable Twilio Media Streams (WebSocket audio feed from caller)
	• Receive raw 8kHz mulaw audio chunks on your WebSocket endpoint
	• Send audio chunks to Deepgram real-time STT — get live transcript
	• Build system prompt: define agent persona and use case
	• Send transcript to LLM (GPT-4o, Claude, or Groq) and log the reply
	• Handle empty/silence chunks — add a fallback prompt
	✓ **Milestone:** User asks a question → terminal logs a correct LLM text reply.
	⚡ **Alt:** Deepgram alternative: OpenAI Whisper (self-host with faster-whisper, free) or AssemblyAI (5 hrs free, REST-only, adds latency).

STT Integration Note

Twilio sends audio as 8kHz mulaw PCM. Deepgram accepts this format natively. If using Whisper, convert first: audioop.ulaw2lin() + audioop.ratecv() to 16kHz before passing to the model.

Phase 3 — Voice Response System (TTS)

Complete the full speech loop. The agent converts its text response back to natural-sounding audio and streams it to the caller in real time.

Day 3	**Text-to-Speech + Full Loop**
	Goal: Full speech → LLM → speech loop. Caller hears a natural voice reply.
	• Integrate ElevenLabs TTS API (10k chars/month free)
	• Convert LLM text response to MP3 / mulaw audio
	• Stream audio back through Twilio Media Stream to the caller
	• Add opening greeting message on call connect
	• Add silence fallback: "Sorry, I didn't catch that — could you repeat?"
	• Add conversation memory: pass last 3–5 turns to LLM in each request
	✓ **Milestone:** Caller asks a question and hears a natural voice reply. Full conversation loop works.
	⚡ **Alt:** ElevenLabs alternative: Google TTS (1M chars/month free) or Amazon Polly (5M chars/year free, no credit card for trial).

Conversation Memory Pattern

```
history = []  # Persists per call session
history.append({'role': 'user', 'content': transcript})
response = llm.chat(messages=history[-6:])  # last 3 turns
history.append({'role': 'assistant', 'content': response})
```

Phase 4 — Testing, Stability & Deployment

Make the demo reliable and presentable. This phase combines edge-case testing with production deployment so the agent runs on a real public number.

Day 4	**Testing + Deployment**
	Goal: Demo works consistently. Public number is live and ready for presentation.
	• Run all 6 minimum tests (see table below)
	• Handle edge cases: background noise, long pauses, very long queries
	• Add conversation logging to file or Supabase (free tier)
	• Add graceful call-end handling and error fallbacks
	• Deploy backend to Render or Railway (free tier)
	• Update Twilio webhook to production URL
	• Run real phone tests from at least 2 different devices
	• Prepare demo script: 3–4 questions the caller will ask
	✓ **Milestone:** Anyone can call the public number and have a smooth, working AI conversation.
	⚡ **Alt:** Render alternative: Railway (free $5 credit, simpler deploys) or Fly.io (free tier, better global latency).

Minimum Tests for Demo Readiness

Run these 6 tests before considering the demo presentable. All must pass consistently across at least 3 back-to-back calls.

#	Test	How to Test	Alt Method
1	Call connects within 3 seconds	Twilio call logs	ngrok request inspector
2	User speech transcribed correctly	Log transcript to console	Deepgram dashboard
3	LLM generates correct response	Inspect API response JSON	Add test prompts script
4	Voice output plays clearly	Listen on test call	Record & playback locally
5	Agent handles silence gracefully	Pause during test call	Send empty audio chunk
6	Call ends without crashing	Hang up mid-response	Check server error logs

Timeline Summary

Total build time: 20–25 hours across 4 days. Each day is a self-contained phase with a clear milestone.

Phase	Timeline	Primary Stack	Alternative Stack
Environment Setup	Day 1	FastAPI + Twilio webhook	Node.js + Express
Speech Pipeline (STT)	Day 2	Deepgram	OpenAI Whisper
Voice Response (TTS)	Day 3	ElevenLabs	Google TTS / Amazon Polly
Testing + Stability	Day 4	Manual call tests	Automated test scripts

✅ Total: ~20–25 hours · 4 days · $0 (free-tier stack)

Tips & Common Pitfalls

Audio Format

- Twilio sends 8kHz mulaw — always check your STT service accepts this format
- Deepgram handles it natively; Whisper needs conversion via audioop

Latency

- LLM response time is the main bottleneck — use streaming responses where possible
- Groq offers the lowest latency free LLM inference (~100ms vs ~800ms for GPT-4)
- ElevenLabs streaming TTS reduces time-to-first-audio significantly

Demo Script

- Prepare 3–4 specific questions your demo caller will ask — test all of them beforehand
- Hardcode a graceful answer for anything outside scope: “I’m only able to help with X today.”

Deployment

- Always test on the deployed URL before a live demo — local ngrok behaves differently
- Set Render service to ‘Always On’ to avoid cold start delays on first call